Deep Reinforcement Learning

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
Homepage

April 2026

Why Deep RL?

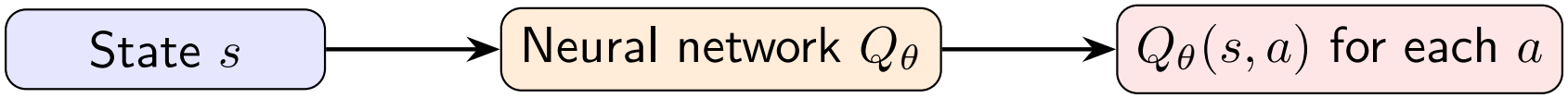
Tabular Q-learning stores $Q(s, a)$ for every state–action pair.

This fails when:

- states are continuous (e.g., joint angles, pixel images)
- the state space is enormous (e.g., 10^{170} positions in Go)

Why Deep RL?

Key idea: approximate $Q(s, a)$ with a deep neural network $Q_\theta(s, a)$.



The network *generalizes* across similar states.

Deep Q-Networks (DQN)

The target Q-value for a transition (s, a, r, s'):

$$y = r + \gamma \max_{a'} Q_\theta(s', a')$$

Train Q_θ to minimize the squared TD error:

$$\mathcal{L}(\theta) = \big(Q_\theta(s, a) - y\big)^2$$

Deep Q-Networks (DQN)

<u>The moving target problem</u>

We do not have Q^*. We have a parameterised approximation Q_θ. So we try to make Q_θ satisfy the Bellman equation by minimising:

$$\mathcal{L}(\theta) = \left(Q_\theta(s,a) - \underbrace{\left[r + \gamma \max_{a'} Q_\theta(s',a') \right]}_{y(\theta)} \right)^2$$

Deep Q-Networks (DQN)

We do not have Q^*. We have a parameterised approximation Q_θ. So we try to make Q_θ satisfy the Bellman equation by minimising:

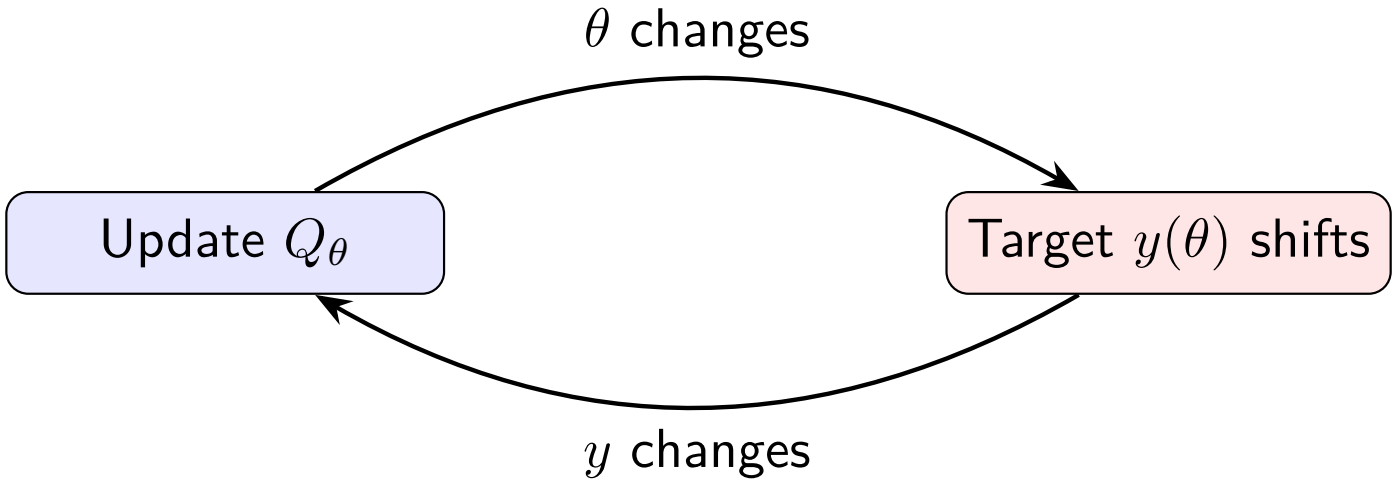
$$\mathcal{L}(\theta) = \left(Q_\theta(s,a) - \underbrace{\left[r + \gamma \max_{a'} Q_\theta(s',a') \right]}_{y(\theta)} \right)^2$$

The problem:

Both the prediction $Q_\theta(s,a)$ and the target $y(\theta)$ depend on θ.

When you take a gradient step to reduce the error, you move θ, which moves the prediction *and* the target simultaneously. You are trying to close a gap that keeps shifting.

Deep Q-Networks (DQN)



We will see how to solve this next. But there is one more problem.

Problem 1: correlated samples

An agent acts sequentially: frame 1, frame 2, frame 3 ...

Consecutive transitions are highly correlated — the network overfits to the recent sequence and forgets earlier experiences. SGD assumes i.i.d. samples; sequential RL data violates this badly.

Problem 1: correlated samples

Fix: experience replay:

- store every transition (s, a, r, s') in a large buffer $\mathcal{D}$
- at each training step, sample a *random* mini-batch from $\mathcal{D}$
- random sampling breaks correlations $\rightarrow$ data is approximately i.i.d.
- each transition is reused many times $\rightarrow$ better sample efficiency

Problem 2: the moving target

In Q-learning, the training target is:

$$y(\theta) = r + \gamma \max_{a'} Q_\theta(s', a')$$

If the *same* network computes both the prediction and the target, every weight update shifts $y(\theta)$ — like a dog chasing its own tail. This causes oscillation and Q-value explosion.

Problem 2: the moving target

Fix: target network:

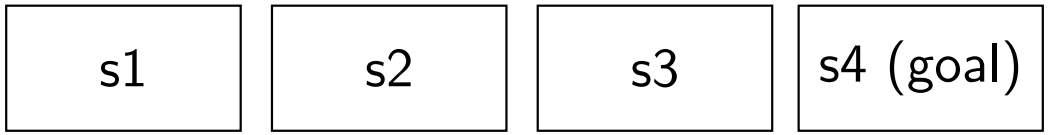
- maintain two networks with identical architecture
- online network Q_θ: updated at every step
- target network Q_{θ^-}: weights frozen, synced every C steps
- compute targets using Q_{θ^-}:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$$

- y is now fixed for C steps $\rightarrow$ stable, consistent gradient direction

Setup: 1D corridor

A simple corridor with 4 positions:



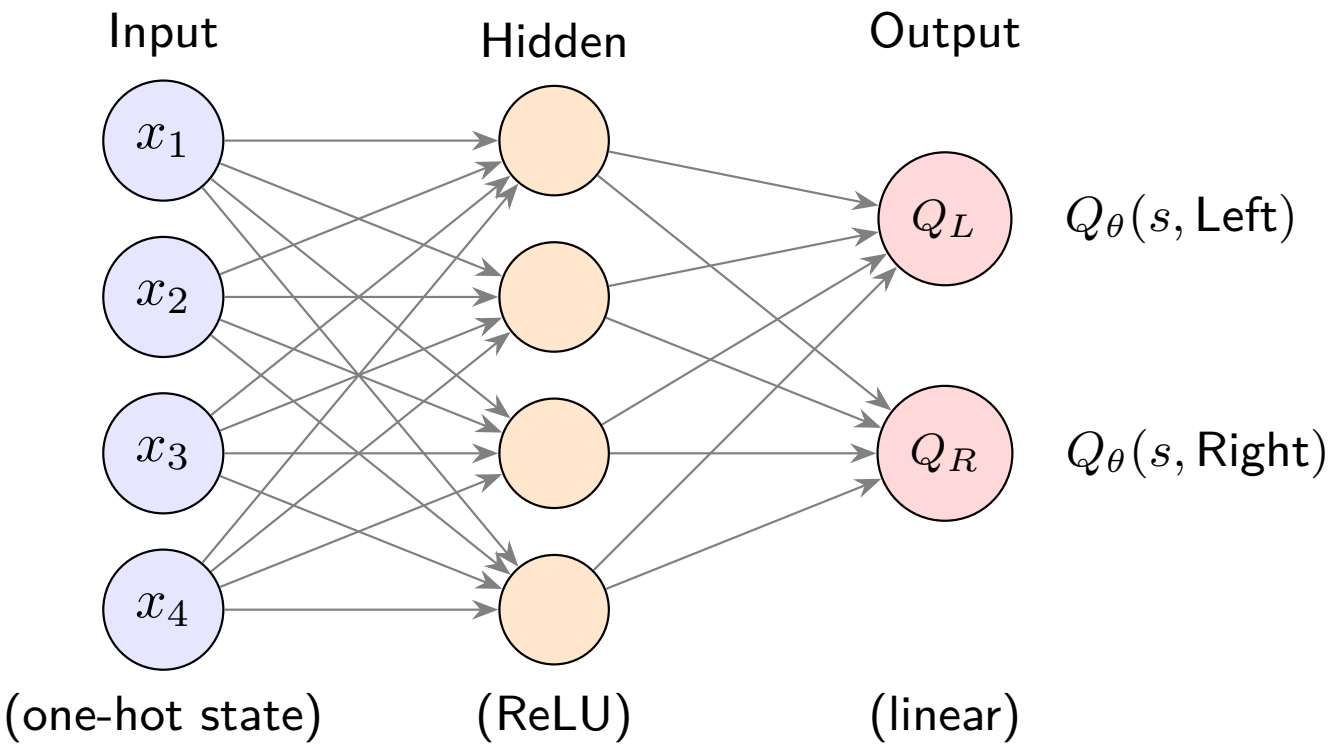
- Actions: Left, Right
- Reward: $+10$ on reaching $s4$, -1 otherwise
- $\gamma = 0.9$, learning rate $\alpha = 0.5$

Network: since states are just indices, we embed each state as a one-hot vector and pass it through a tiny network. For illustration, we track $Q_\theta(s, a)$ as a table that gets updated via gradient steps.

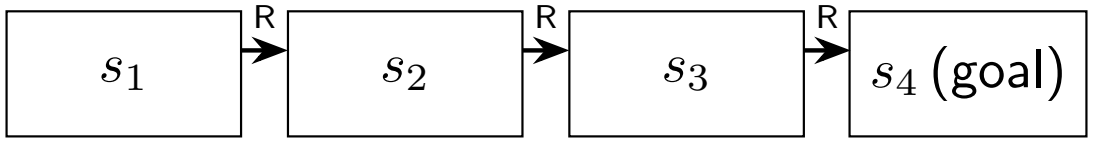
Setup: 1D corridor

NN for the corridor example

State s is one-hot encoded: $s2 = [0, 1, 0, 0]^\top$.



Detailed example: 1D corridor



- Actions: Left, Right
- Reward: $r = +10$ on reaching s_4; $r = -1$ otherwise
- $\gamma = 0.9$, learning rate $\alpha = 0.5$
- Agent always takes **Right** (greedy path for clarity)
- Batch size $= 1$ (one update per transition, for illustration)
- Target network synced every $C = 3$ steps (end of each episode)

Detailed example: 1D corridor

Initial Q-table (both online Q_θ and target Q_{θ^-}):

State	$Q(s, \text{Left})$	$Q(s, \text{Right})$
s_1	0	0
s_2	0	0
s_3	0	0
s_4	0	0

Episode 1: discovering the goal

Step 1: $s_1 \xrightarrow{\text{Right}} s_2, \quad r = -1$

Network input	one-hot $s_1 = [1, 0, 0, 0]$
Network output	$Q_\theta(s_1, \mathsf{L}) = 0, \quad Q_\theta(s_1, \mathsf{R}) = 0$
Target network	$\max_{a'} Q_{\theta^-}(s_2, a') = \max(0, 0) = 0$
Target	$y = -1 + 0.9 \times 0 = -1.0$
Prediction	$Q_\theta(s_1, \mathsf{R}) = 0$
TD error	$\delta = 0 - (-1.0) = 1.0$
Loss	$\delta^2 = 1.0$
Update	$Q_\theta(s_1, \mathsf{R}) \leftarrow 0 + 0.5 \times (-1.0 - 0) = -0.5$

Episode 1: discovering the goal

Step 2: $s_2 \xrightarrow{\text{Right}} s_3, \quad r = -1$

Network input	one-hot $s_2 = [0, 1, 0, 0]$
Network output	$Q_\theta(s_2, \mathsf{L}) = 0, \quad Q_\theta(s_2, \mathsf{R}) = 0$
Target network	$\max_{a'} Q_{\theta^-}(s_3, a') = \max(0, 0) = 0$
Target	$y = -1 + 0.9 \times 0 = -1.0$
Prediction	$Q_\theta(s_2, \mathsf{R}) = 0$
TD error	$\delta = 0 - (-1.0) = 1.0$
Loss	$\delta^2 = 1.0$
Update	$Q_\theta(s_2, \mathsf{R}) \leftarrow 0 + 0.5 \times (-1.0 - 0) = -0.5$

Episode 1: discovering the goal

Step 3: $s_3 \xrightarrow{\text{Right}} s_4, \quad r = +10$

Network input	one-hot $s_3 = [0, 0, 1, 0]$
Network output	$Q_\theta(s_3, \mathsf{L}) = 0, \quad Q_\theta(s_3, \mathsf{R}) = 0$
Target network	$\max_{a'} Q_{\theta^-}(s_4, a') = \max(0, 0) = 0$
Target	$y = 10 + 0.9 \times 0 = 10.0$
Prediction	$Q_\theta(s_3, \mathsf{R}) = 0$
TD error	$\delta = 0 - 10.0 = -10.0$
Loss	$\delta^2 = 100.0$
Update	$Q_\theta(s_3, \mathsf{R}) \leftarrow 0 + 0.5 \times (10.0 - 0) = +5.0$

Episode 1: discovering the goal

Target network sync ($\theta^- \leftarrow \theta$ after $C = 3$ steps):

State	$Q_{\theta^-}(s, \text{Left})$	$Q_{\theta^-}(s, \text{Right})$
s_1	0	-0.5
s_2	0	-0.5
s_3	0	$+5.0$

Only s_3 has a positive Q-value. The reward signal has not yet propagated to s_1 or s_2.

Episode 2: reward propagates one step back

Step 4: $s_1 \xrightarrow{\text{Right}} s_2, \quad r = -1$

Target network	$\max_{a'} Q_{\theta^-}(s_2, a') = \max(0, -0.5) = 0$
	(Left Q-value wins the max — Right is negative)
Target	$y = -1 + 0.9 \times 0 = -1.0$
Prediction	$Q_\theta(s_1, \text{R}) = -0.5$
Loss	$(-0.5 - (-1.0))^2 = 0.25$
Update	$Q_\theta(s_1, \text{R}) \leftarrow -0.5 + 0.5 \times (-1.0 - (-0.5)) = -0.75$

s_1 gets worse — the positive reward at s_4 has not reached it yet.

Episode 2: reward propagates one step back

Step 5: $s_2 \xrightarrow{\text{Right}} s_3, \quad r = -1$

Target network	$\max_{a'} Q_{\theta^-}(s_3, a') = \max(0, 5.0) = 5.0$
	(now uses the updated target network)
Target	$y = -1 + 0.9 \times 5.0 = 3.5$
Prediction	$Q_\theta(s_2, \mathsf{R}) = -0.5$
TD error	$\delta = -0.5 - 3.5 = -4.0$
Loss	$\delta^2 = 16.0$
Update	$Q_\theta(s_2, \mathsf{R}) \leftarrow -0.5 + 0.5 \times (3.5 - (-0.5)) = +1.5$

s_2 now has a positive Q-value — reward propagated one step back!

Episode 2: reward propagates one step back

Step 6: $s_3 \xrightarrow{\text{Right}} s_4, \quad r = +10$

Target	$y = 10 + 0.9 \times 0 = 10.0$
Prediction	$Q_\theta(s_3, \text{R}) = 5.0$
Loss	$(5.0 - 10.0)^2 = 25.0$
Update	$Q_\theta(s_3, \text{R}) \leftarrow 5.0 + 0.5 \times (10.0 - 5.0) = +7.5$

Target network sync ($\theta^- \leftarrow \theta$):

State	$Q_{\theta^-}(s, \text{Left})$	$Q_{\theta^-}(s, \text{Right})$
s_1	0	-0.75
s_2	0	$+1.5$
s_3	0	$+7.5$

Episode 3: reward reaches s_1

Step 7: $s_1 \xrightarrow{\text{Right}} s_2, \quad r = -1$

Target network	$\max_{a'} Q_{\theta^-}(s_2, a') = \max(0, 1.5) = 1.5$
	(Right now wins the max — s_2 has learned)
Target	$y = -1 + 0.9 \times 1.5 = 0.35$
Prediction	$Q_\theta(s_1, \text{R}) = -0.75$
Loss	$(-0.75 - 0.35)^2 = 1.21$
Update	$Q_\theta(s_1, \text{R}) \leftarrow -0.75 + 0.5 \times (0.35 - (-0.75)) = -0.20$

Still negative at s_1, but moving in the right direction.

Episode 3: reward reaches s_1

Step 8: $s_2 \xrightarrow{\text{Right}} s_3, \quad r = -1$

Target	$y = -1 + 0.9 \times 7.5 = 5.75$
Prediction	$Q_\theta(s_2, \text{R}) = 1.5$
Update	$Q_\theta(s_2, \text{R}) \leftarrow 1.5 + 0.5 \times (5.75 - 1.5) = +3.63$

Step 9: $s_3 \xrightarrow{\text{Right}} s_4, \quad r = +10$

Target	$y = 10.0$
Prediction	$Q_\theta(s_3, \text{R}) = 7.5$
Update	$Q_\theta(s_3, \text{R}) \leftarrow 7.5 + 0.5 \times (10 - 7.5) = +8.75$

Target network sync:

Episode 3: reward reaches s_1

State	$Q_{\theta^-}(s, \text{Left})$	$Q_{\theta^-}(s, \text{Right})$
s_1	0	-0.20
s_2	0	$+3.63$
s_3	0	$+8.75$

Episode 4: policy correct everywhere

Step 10: $s_1 \xrightarrow{\text{Right}} s_2, \quad r = -1$

Target network	$\max_{a'} Q_{\theta^-}(s_2, a') = \max(0, 3.63) = 3.63$
Target	$y = -1 + 0.9 \times 3.63 = 2.27$
Prediction	$Q_\theta(s_1, \text{R}) = -0.20$
Update	$Q_\theta(s_1, \text{R}) \leftarrow -0.20 + 0.5 \times (2.27 - (-0.20)) = +1.04$

$Q(s_1, \text{Right})$ is finally positive. The policy now points Right from every state.

Episode 4: policy correct everywhere

Q-values across episodes:

	$Q(s_1, \mathsf{R})$	$Q(s_2, \mathsf{R})$	$Q(s_3, \mathsf{R})$
After ep. 1	-0.50	-0.50	$+5.00$
After ep. 2	-0.75	$+1.50$	$+7.50$
After ep. 3	-0.20	$+3.63$	$+8.75$
After ep. 4	$+1.04$	$+5.26$	$+9.38$

Observation: the reward propagates *one state per episode*. This is because the target network syncs once per episode — each sync carries the signal one step further back toward s_1.

Directly learning the policy

DQN learns $Q_\theta(s, a)$ and derives the policy as $\arg\max_a Q_\theta(s, a)$.

Limitations:

- $\arg\max$ requires a discrete action space
- cannot represent stochastic policies

Directly learning the policy

Policy gradient idea: directly parameterize $\pi_\theta(a|s)$ and optimize

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t r_t\right]$$

The policy gradient theorem (Williams, 1992) gives:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]$$

where $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ is the return from step t.

Update rule and intuition

REINFORCE update:

$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Intuition:

- $\nabla_\theta \log \pi_\theta(a_t|s_t)$: direction that increases probability of a_t
- multiply by G_t: scale by how good the outcome was
- if $G_t > 0$: make a_t more likely
- if $G_t < 0$: make a_t less likely

Network: takes state s as input, outputs $\pi_\theta(a|s)$ (softmax over actions). Gradient computed by backpropagation through $\log \pi_\theta$.

Setup: one state, two actions

One state s. Two actions: Left, Right. True rewards unknown to the agent.

Softmax policy with parameters θ_L, θ_R:

$$\pi(\text{Right}|s) = \frac{e^{\theta_R}}{e^{\theta_L} + e^{\theta_R}}, \qquad \pi(\text{Left}|s) = \frac{e^{\theta_L}}{e^{\theta_L} + e^{\theta_R}}$$

Initialize $\theta_L = 0,\ \theta_R = 0$, so:

$$\pi(\text{Right}|s) = \pi(\text{Left}|s) = 0.5$$

Learning rate $\alpha = 0.1$.

Trial 1: action Right, reward $+10$

Gradients of $\log \pi(\text{Right}|s)$:

$$\nabla_{\theta_R} \log \pi(\text{Right}) = 1 - \pi(\text{Right}) = 0.5$$

$$\nabla_{\theta_L} \log \pi(\text{Right}) = -\pi(\text{Left}) = -0.5$$

Update with $G = r = 10$:

$$\theta_R = 0 + 0.1 \times 10 \times 0.5 = +\mathbf{0.5}$$

$$\theta_L = 0 + 0.1 \times 10 \times (-0.5) = -\mathbf{0.5}$$

New probabilities:

$$\pi(\text{Right}) = \frac{e^{0.5}}{e^{0.5} + e^{-0.5}} \approx \mathbf{0.73}, \qquad \pi(\text{Left}) \approx 0.27$$

The agent now prefers Right.

Trial 2: action Left, reward -1

With current probabilities: $\pi(\text{Right}) = 0.73$, $\pi(\text{Left}) = 0.27$

Gradients of $\log \pi(\text{Left}|s)$:

$$\nabla_{\theta_L} \log \pi(\text{Left}) = 1 - \pi(\text{Left}) = 0.73$$

$$\nabla_{\theta_R} \log \pi(\text{Left}) = -\pi(\text{Right}) = -0.73$$

Update with $G = r = -1$:

$$\theta_L = -0.5 + 0.1 \times (-1) \times 0.73 = \mathbf{-0.573}$$

$$\theta_R = +0.5 + 0.1 \times (-1) \times (-0.73) = \mathbf{+0.573}$$

$\pi(\text{Right})$ increases further. Over many trials, the policy converges toward Right.

Summary

	DQN	**Policy gradient**
Learns	$Q_\theta(s, a)$	$\pi_\theta(a\|s)$
Policy	implicit ($\arg\max$)	explicit
Action space	discrete	discrete or continuous
Key trick	replay + target net	$\log \pi$ gradient

Summary

- Neural networks let RL scale to large/continuous state spaces
- DQN: approximate Q^* with a network; stabilized by replay and target network
- Policy gradient: directly optimize π_θ using reward-weighted log gradients
- Actor-critic combines both: lower variance, faster learning

Mnih et al. (2015) *Nature* 518.
Williams (1992) *Mach. Learn.* 8.
Sutton & Barto (2018) *RL: An Introduction*.